Summary

AI Agents for Application Security Review

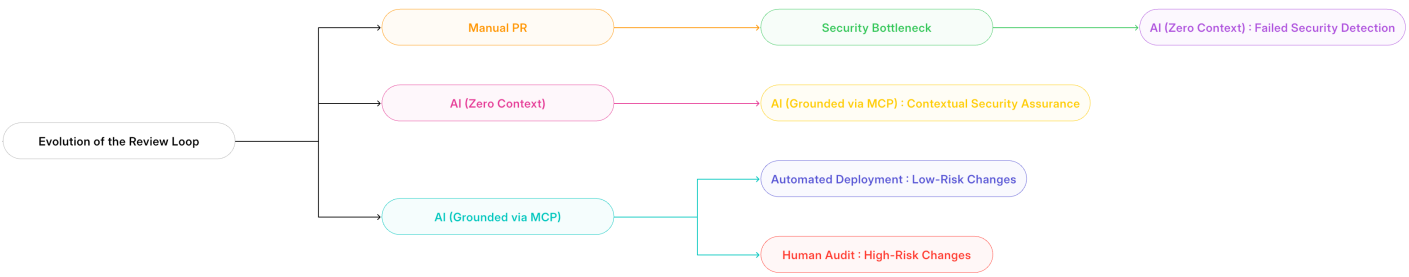
This technical briefing explores the integration of AI agents into the Application Security (AppSec) lifecycle, transitioning from ad hoc manual reviews to context-aware automated governance. Daniel (Architect) and Magnus Johnsson (AppSec Specialist) demonstrate that while vanilla AI agents fail to detect logic-based vulnerabilities, they become highly effective when grounded with organizational context and industry standards like OWASP.

The BOLA Case Study: Why Context Matters

A core demonstration centered on a "Broken Object Level Authorization" (BOLA) vulnerability within a new `getById` endpoint.

- **Manual Review:** A specialist immediately identified that the code lacked a "market concept" check, allowing any authenticated user to access products from any market.
- **Zero-Context AI:** GitHub Copilot failed to identify the security flaw, focusing instead on superficial style and pattern critiques.
- **Context-Grounded AI:** Once the agent was provided with a specific instruction ("private API endpoints must verify market-level access"), it successfully flagged the missing authorization check and referenced the instruction as the rationale.

Evolution of the Review Loop



Strategic Implementation: The MCP Approach

To scale security intelligence without confusing the model, the team transitioned from flat Markdown

instructions to a **Model Context Protocol (MCP) server**.

- **Separation of Concerns:** Data is indexed into three distinct silos: Code, Ops Lake, and Compliance. This structure minimized hallucinations that occurred when using a single, massive index.
- **Deterministic Feedback:** By providing agents with the OWASP API Top 10 and the Application Security Verification Standard (ASVS), the agents generated structured security reports rather than vague commentary.
- **Generative Security:** The team demonstrated that a well-instructed agent could proactively write secure code. When prompted to add an "Update" endpoint, the agent correctly inferred the need for a new `can_write_products` permission and implemented the market-check logic Daniel had previously forgotten.

The AppSec Definition of Done

Regardless of whether a review is manual or automated, the team proposed eight critical verification questions for every code change:

1. **Authentication/Authorization:** Does the request require a valid session and specific permission scopes?
2. **Input Validation:** Are values validated as domain primitives to prevent injection?
3. **Output Decoding:** Is data properly encoded to prevent XSS or leakage?
4. **Dependencies:** Are third-party integrations patched and secure?
5. **Credentials:** Are there any hardcoded keys or production data in the repository?
6. **Business Logic Misuse:** Can the logic be manipulated to bypass intended constraints?
7. **Logging & Errors:** Is the error handling sufficient without leaking internal exception details?
8. **Mega Temp Tables:** Are temporary data structures handled securely?

Governance & Accountability

The transition to AI-assisted security does not remove the need for human oversight.

- **Human-in-the-Loop:** Humans remain the point of accountability, particularly for high-value functions or regulatory requirements under the EU AI Act.
- **Risk-Based Triage:** The ultimate goal is a software factory where low-risk PRs are fully automated, while high-risk logic changes trigger mandatory multi-human reviews based on AI-generated risk classifications.

Security Integration Roadmap

PHASE 1

Manual Review

Ad hoc, high friction, specialist dependent.

PHASE 2

Simple Instructions

Basic grounding; catch obvious logic flaws.

PHASE 3

MCP Integration

Scalable, multi-index context (Security + Compliance).

PHASE 4

Generative AppSec

AI writes secure-by-default code based on intent.

PHASE 5

Risk-Based Governance

Automated promotion vs. human accountability loops.